
LÓGICA MATEMÁTICA

Cuarto del Grado en Matemáticas

Hugo Marquerie

Profesor: Arturo Rodríguez Fanlo
Facultad de Ciencias - Universidad Autónoma de Madrid
Cuatrimestre

27 de enero de 2026

Puedes encontrar mi colección completa de apuntes en <https://hmarquer.github.io>.

Índice

1. Teoría de modelos

1.1 Lenguajes y estructuras

1.1.1 Lenguajes y variables

Definición 1.1.1 (Lenguaje de primer orden). L es un lenguaje de primer orden (FOL) $\iff L$ es un conjunto de símbolos tal que existe una partición de L en:

- Símbolos formales $\{\dot{=}, \top, \neg, \wedge, \exists\}$.
- Símbolos de función: $F = \bigcup_{k=0}^{\infty} F_k$, donde $\forall k \in \mathbb{N} \cup \{0\} : F_k$ es un conjunto de símbolos de función de aridad k .
- Símbolos de relación: $R = \bigcup_{k=0}^{\infty} R_k$, donde $\forall k \in \mathbb{N} \cup \{0\} : R_k$ es un conjunto de símbolos de relación de aridad k .

Definición 1.1.2 (Constante). Sea L un FOL, $c \in L$ es una constante $\iff c$ es un símbolo de función de aridad 0 en L .

Definición 1.1.3 (Proposición). Sea L un FOL, $p \in L$ es una proposición $\iff p$ es un símbolo de relación de aridad 0 en L .

Definición 1.1.4 (Signatura). Sea L un FOL, la signatura de L es $\bigsqcup_k F_k \sqcup \bigsqcup_k R_k \subset L$.

Observación 1.1.5. Un lenguaje de primer orden L queda completamente determinado por su signatura. Por ello, en general, se identifica un lenguaje con su signatura.

Ejemplos 1.1.1 (de lenguajes de primer orden).

- 1 Lenguaje vacío: la signatura es vacía, es decir, solo tenemos los símbolos formales.
- 2 Lenguaje de grupos: viene dado por la signatura $L_{\text{gr}} = \{\cdot, 1, \cdot^{-1}\}$ donde $\cdot$ es un símbolo de función binario, 1 es una constante y $\cdot^{-1}$ es un símbolo de función unario.

Alternativamente, el lenguaje aditivo de grupos es $L_{\text{gr}}^+ = \{+, 0, -\}$.

- 3 El lenguaje de anillos (y cuerpos) es $L_a = \{+, \cdot, -, 0, 1\}$ donde $+$ y $\cdot$ son símbolos de función binarios, $-$ es un símbolo de función unario y 0 y 1 son constantes.
- 4 El lenguaje de órdenes es $L_{<} = \{<\}$ donde $<$ es un símbolo de relación binario.

- [5] El lenguaje de grafos es $L_E = \{E\}$ donde E es un símbolo de relación binario.
 - [6] El lenguaje de equivalencia es $L_{\sim} = \{\sim\}$ donde $\sim$ es un símbolo de relación binario.
 - [7] Dado un cuerpo K , el lenguaje de K -espacios vectoriales es $L_{K\text{-vec}} = \{+, -, 0\} \cup \{k \cdot\}_{k \in K}$ donde $+$ es un símbolo de función binario, $-$ es un símbolo de función unario, 0 es una constante y $\forall k \in K : k \cdot$ es un símbolo de función unario.
 - [8] El lenguaje de la aritmética es $L_{\mathbb{N}} = \{\cdot, +, S, 0, <\}$ donde $\cdot$ y $+$ son símbolos de función binarios, S es un símbolo de función unario, 0 es una constante y $<$ es un símbolo de relación binario.
- El lenguaje de la aritmética con exponencial es la extensión de $L_{\mathbb{N}}$ dada por la signatura $L_{\mathbb{N}, \exp} = \{\cdot, +, \cdot, S, 0, <\}$ donde $\cdot$ es un símbolo de función binario (elevar a la potencia).
- [9] El lenguaje de los reales es el lenguaje extensión del lenguaje de anillos dado por la signatura $L_{\mathbb{R}} = \{+, \cdot, -, 0, 1, <\}$ donde $<$ es un símbolo de relación binario.
 - [10] El lenguaje de conjuntos es $L_{\in} = \{\in\}$ donde $\in$ es un símbolo de relación binario.

Es importante remarcar que los elementos de un lenguaje son solo símbolos sin significado alguno (excepto por la distinción formal entre el tipo de símbolo y su aridad). Dicho esto, es evidente que al construir un lenguaje tenemos en mente un uso y, por tanto, elegimos los símbolos de acuerdo con las interpretaciones previstas. Sin embargo, debemos tener presente que estas pretendidas interpretaciones (las cuales definiremos más adelante al hablar de “estructuras”) no están en el lenguaje en sí mismo.

Por ejemplo, conviene notar que, como lenguajes, L_{gr} y L_{agr} son (esencialmente) el mismo lenguaje, en el sentido de que hay una correspondencia biyectiva que preserva los tipos de símbolos y las aridades. En efecto, aunque tengamos en mente utilizar L_{agr} para grupos abelianos y L_{gr} para grupos arbitrarios, los lenguajes en sí mismos no indican esta distinción.

Más aún, los lenguajes $L_{<}$, L_E , $L_{\sim}$ y $L_{\in}$ son esencialmente idénticos. En efecto, aunque implícitamente tenemos en mente utilizar $L_{<}$ para órdenes, L_E para grafos, $L_{\sim}$ para relaciones de equivalencia y $L_{\in}$ para conjuntos, los lenguajes en sí mismos no indican que $<$ sea un orden, E sea una relación de grafo, $\sim$ sea una relación de equivalencia o $\in$ sea la relación de pertenencia. En lo relativo al lenguaje, solo tenemos símbolos formales sin significado.

Definición 1.1.6 (Conjunto de variables). Sea L un lenguaje de primer orden, X es un conjunto de variables para $L \iff X$ es un conjunto disjunto de L .

- A cada subconjunto de X se le llama **variable**.

- A cada variable se le dice **simple** si es un conjunto unitario.

1.1.2 Términos y fórmulas

Definición 1.1.7 (Palabra). Sea L un lenguaje de primer orden con variables X , una palabra en L es una tupla finita cuyos elementos son símbolos de $L \cup X$.

Definición 1.1.8 (Término). Sea L un lenguaje de primer orden con variables X , los términos de L son las palabras definidas inductivamente como sigue:

- Caso 0: $\text{Ter}_0(L; X)$ es el conjunto de las constantes y variables simples.
- Caso $n + 1$: $\text{Ter}_{n+1}(L; X)$ es el conjunto que incluye a $\text{Ter}_n(L; X)$ y contiene las palabras de la forma $f t_1 \dots t_k$ donde f es un símbolo de función k -ario y $t_1, \dots, t_k \in \text{Ter}_n(L; X)$.

El conjunto de términos de L se denota por $\text{Ter}(L; X) := \bigcup_{n=0}^{\infty} \text{Ter}_n(L; X)$.

Definición 1.1.9 (Fórmula). Sea L un lenguaje de primer orden con variables X , una fórmula de L es una palabra en $L \cup X$ definida inductivamente como sigue:

- Caso 0: $\text{For}_0(L; X)$ es el conjunto de las palabras de la forma $R t_1 \dots t_m$ donde R es un símbolo de relación de aridad m y $t_1, \dots, t_m \in \text{Ter}(L; X)$. Además, incluye la palabra $\top$ y las palabras de la forma $t_1 t_2$ donde $t_1, t_2 \in \text{Ter}(L; X)$.
- Caso $n + 1$: $\text{For}_{n+1}(L; X)$ es el conjunto que incluye a $\text{For}_n(L; X)$ y contiene las palabras de la forma:
 - (i) $\wedge \varphi_1 \varphi_2$ donde $\varphi_1, \varphi_2 \in \text{For}_n(L; X)$.
 - (ii) $\neg \varphi$ donde $\varphi \in \text{For}_n(L; X)$.
 - (iii) $\exists x \varphi$ donde x es una variable simple y $\varphi \in \text{For}_n(L; X)$.

El conjunto de fórmulas de L se denota por $\text{For}(L; X) := \bigcup_{n \in \mathbb{N}} \text{For}_n(L; X)$.

Ejemplo 1.1.2 (de fórmula). En $L_{\mathbb{R}}$, $\exists x \exists y \wedge \neg \wedge = \cdot x x y < x 0 \top$ es una fórmula. Normalmente, se escribiría como $\exists x, y : (\neg(x^2 = y \wedge x < 0) \wedge \top)$.

Aunque, evidentemente, tenemos un significado en mente a la hora de definir las fórmulas y los términos, por ahora no hemos definido en absoluto qué es eso del “significado” de una fórmula o de un término. En lo que se refiere al lenguaje, las fórmulas y los términos son solo cadenas de símbolos formales construidas siguiendo una serie de reglas sintácticas.

Observación 1.1.10. Si consideramos el caso con $\mathbf{X} = \emptyset$ y $\mathbf{L}$ un lenguaje proposicional (es decir, que su signatura solo contiene símbolos de relación 0-arios), la lógica de primer orden con $\mathbf{L}$ se reduce a la lógica proposicional, es decir, $\text{Ter}(\mathbf{L}; \mathbf{X}) = \emptyset$ y $\text{For}(\mathbf{L}; \mathbf{X})$ son las fórmulas en lógica proposicional. De este modo, la lógica de primer orden es una generalización de la lógica proposicional, también llamada lógica de orden cero.

Notación. Toda biyección entre dos conjuntos de variables induce una biyección entre los respectivos conjuntos de términos y fórmulas. Además, como los términos y las fórmulas utilizan solo un número finito de variables, en general podemos asumir que trabajamos con un número numerable de variables. En consecuencia, a la hora de definir los términos y fórmulas es esencialmente irrelevante la forma concreta del conjunto de variables. Es por ello que, salvo que sea estrictamente necesario, evitaremos especificar explícitamente la familia de variables $\mathbf{X}$ que utilizamos. Por tanto, a partir de ahora, normalmente escribiremos $\text{Ter } \mathbf{L} := \text{Ter}(\mathbf{L}, \mathbf{X})$ y $\text{For } \mathbf{L} := \text{For}(\mathbf{L}, \mathbf{X})$.

1.1.3 Notación polaca y notación estándar

Por simplicidad, usamos solo los símbolos $\top$ (tautología), $\neg$ (negación), $\wedge$ (conjunción) y $\exists$ (existe) para las definiciones. Dicho esto, en la práctica, también usamos los símbolos $\perp$ (falso), $\vee$ (disyunción), $\rightarrow$ (condicional), $\leftrightarrow$ (equivalencia) y $\forall$ (universal). Estos símbolos los introducimos como abreviaturas del siguiente modo: Escribimos

- $\perp$ para $\neg \top$,
- $\varphi \vee \psi$ para $\neg(\neg\varphi \wedge \neg\psi)$,
- $\varphi \rightarrow \psi$ para $\neg\varphi \vee \psi$,
- $\varphi \leftrightarrow \psi$ para $(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$, y
- $\forall x \varphi$ para $\neg \exists x \neg \varphi$.

Por otro lado, en las definiciones formales de términos y fórmulas estamos utilizando la notación polaca (que escribe las funciones, relaciones y conectivas como prefijos). Esta notación tiene la virtud de evitar el uso de paréntesis, pero resulta poco legible para los humanos. Para facilitar la lectura utilizamos la notación estándar: Escribimos

- $f(t_1, \dots, t_m)$ para $f t_1 \dots t_m$ y $(t_1 f t_2)$ para $f t_1 t_2$ si f es binario;
- $R(t_1, \dots, t_m)$ para $R t_1 \dots t_m$ y $(t_1 R t_2)$ para $R t_1 t_2$ si R es binario;
- $(t_1 \doteq t_2)$ para $\doteq t_1 t_2$ y usamos $=$ en lugar de $\doteq$ si no hay ambigüedad;

- $(\varphi \wedge \psi)$ para $\wedge \varphi \psi$, $(\varphi \vee \psi)$ para $\vee \varphi \psi$, $(\varphi \rightarrow \psi)$ para $\rightarrow \varphi \psi$ y $(\varphi \leftrightarrow \psi)$ para $\leftrightarrow \varphi \psi$;
- $(\exists x_1, \dots, x_n \varphi)$ para $\exists x_1 \dots \exists x_n \varphi$ y $(\forall x_1, \dots, x_n \varphi)$ para $\forall x_1 \dots \forall x_n \varphi$; y
- $\bigwedge_{i=1}^n \varphi_i$ para $\wedge \dots \wedge \varphi_1 \dots \varphi_n$ y $\bigvee_{i=1}^n \varphi_i$ para $\vee \dots \vee \varphi_1 \dots \varphi_n$.

La notación estándar es ambigua. Por ejemplo, $p \vee q \wedge r$ puede ser $(p \vee q) \wedge r$ o $p \vee (q \wedge r)$. Para subsanar esto utilizamos los paréntesis. Dicho esto, omitimos los paréntesis siempre que la omisión no dé lugar a ambigüedades. En este sentido, para reducir al mínimo el uso de paréntesis, seguiremos el convenio de prioridad $\neg \wedge \exists$, según el cual primero actúa $\neg$, luego actúan las conectivas binarias ($\wedge$, $\vee$, $\rightarrow$ o $\leftrightarrow$), y, por último, los cuantificadores ($\exists$ o $\forall$). De este modo, para la fórmula $\exists x ((\neg \varphi) \wedge \psi)$, podemos escribir simplemente $\exists x \neg \varphi \wedge \psi$.

1.1.3.1 Complejidad

Definición 1.1.11 (Complejidad). Sea L un lenguaje de primer orden, y sea $t \in \text{Ter } L$, $\text{comp}(t)$ es la complejidad de $t \iff \text{comp}(t) = \max\{n \in \mathbb{N} : t \in \text{Ter}_n L\}$.

Sea $\varphi \in \text{For } L$, $\text{comp}(\varphi)$ es la complejidad de $\varphi \iff \text{comp}(\varphi) = \max\{n \in \mathbb{N} : \varphi \in \text{For}_n L\}$

A priori, es posible que una fórmula (o término) se pueda obtener de dos formas distintas a partir de fórmulas (o términos) de menor complejidad. Es decir, en principio, podría darse el caso de que las fórmulas (o los términos) sean ambiguos. Por ejemplo, en notación estándar sin paréntesis, la palabra $\varphi_1 \wedge \varphi_2 \vee \varphi_3$ se puede obtener de dos formas distintas como “ $(\varphi_1 \wedge \varphi_2) \vee \varphi_3$ ” o como “ $\varphi_1 \wedge (\varphi_2 \vee \varphi_3)$ ” (es decir, usando primero $\wedge$ y después $\vee$, o usando primero $\vee$ y después $\wedge$).

Del mismo modo, conviene notar que, en principio, la complejidad de los términos y las fórmulas no está unívocamente definida, es decir, en caso de existir ambigüedades, podría darse la situación de que una fórmula (o término) se pueda obtener de dos formas distintas como fórmula (o término) de complejidad distinta. Por ejemplo, con notación estándar sin paréntesis, la fórmula $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ puede corresponderse con la fórmula $(\varphi_1 \wedge \varphi_2) \wedge (\varphi_3 \wedge \varphi_4)$, de complejidad 3, o con la fórmula $\varphi_1 \wedge (\varphi_2 \wedge (\varphi_3 \wedge \varphi_4))$, de complejidad 4.

A continuación vamos a demostrar que este no es el caso en notación polaca, es decir, que no existen ambigüedades en las fórmulas y los términos tal y como los hemos definido usando notación polaca.

Lema 1.1.1. *Sea L un lenguaje de primer orden, $\eta_1, \dots, \eta_n$ y $\eta'_1, \dots, \eta'_m$ todos términos o todas fórmulas. Entonces, las palabras $\eta_1 \dots \eta_n$ y $\eta'_1 \dots \eta'_m$ son iguales si y solo si $n = m$ y $\eta_1 = \eta'_1, \dots, \eta_n = \eta'_n$.*

Demostración: La implicación de derecha a izquierda es evidente. Demostramos la implicación de izquierda a derecha.

Tenemos que $\eta_1 \dots \eta_n = \eta'_1 \dots \eta'_m$. Sin pérdida de generalidad, asumimos que $m \geq n$. Evidentemente, es suficiente demostrar que $\eta_1 = \eta'_1, \dots, \eta_n = \eta'_n$, pues en tal caso $m = n$. Supongamos por contradicción que $\eta_i \neq \eta'_i$ para algún $i \in \{1, \dots, n\}$ y tomamos $k = \min\{i \in \mathbb{N}_n : \eta_i \neq \eta'_i\}$. Entonces, en particular, $\eta_1 \dots \eta_{k-1} = \eta'_1 \dots \eta'_{k-1}$, luego eliminando los símbolos iniciales, tenemos que $\eta_k \dots \eta_n = \eta'_k \dots \eta'_m$.

Si $|\eta_k| > |\eta'_k|$ (donde $|\cdot|$ indica la longitud de la palabra), eliminando los símbolos finales, tenemos que $\eta_k = \eta'_k \zeta$ con ζ una palabra no vacía. Simétricamente, si $|\eta'_k| > |\eta_k|$, tenemos que $\eta'_k = \eta_k \zeta$ para una palabra ζ no vacía. En consecuencia, es suficiente demostrar lo siguiente:

Afirmación: Ningún fragmento inicial propio de un término es un término. Ningún fragmento inicial propio de una fórmula es una fórmula.

- (a) Términos: Sea η un término y $\eta = \eta' \zeta$ con η' un término. Demostramos por inducción en la longitud de η que $\eta = \eta'$ y $\zeta = \emptyset$. Si $|\eta| = 1$, como η' es un término, tenemos que $1 \leq |\eta'| \leq |\eta| = 1$, luego $|\eta'| = |\eta| = 1$ y $|\zeta| = 0$.

Supongamos que $|\eta| = l > 1$ y se cumple para todo término de longitud menor a l . Como $|\eta| > 1$, tenemos que $\eta = f t_1 \dots t_k$ para cierto símbolo de función f de aridad k ($k \neq 0$) y ciertos términos $t_1, \dots, t_k$. Como $\eta = \eta' \zeta$, concluimos que el primer símbolo de η' es f . Como η' es un término, tenemos que $\eta' = f t'_1 \dots t'_k$ para ciertos términos $t'_1, \dots, t'_k$. Eliminando el primer símbolo, obtenemos que $t_1 \dots t_k = t'_1 \dots t'_k \zeta$.

Supongamos que existe i tal que $t_i \neq t'_i$. Tomamos $m := \min\{i : t_i \neq t'_i\}$. En tal caso, como $t_1 \dots t_{m-1} = t'_1 \dots t'_{m-1}$, eliminando los símbolos iniciales, tenemos que $t_m \dots t_k = t'_m \dots t'_k \zeta$. Si $|t_m| > |t'_m|$, entonces, eliminando los últimos símbolos, tenemos $t_m = t'_m \zeta'$ con cierta palabra ζ' . Ahora bien, $|t_m| < l$, luego $\zeta' = \emptyset$ y $t_m = t'_m$ por hipótesis de inducción. Si por el contrario $|t_m| < |t'_m|$, entonces, eliminando los últimos símbolos, tenemos $t'_m = t_m \zeta'$ con cierta palabra ζ' . Como $|t'_m| < l$, concluimos igualmente que $\zeta' = \emptyset$ y $t_m = t'_m$. En ambos casos, hemos llegado a una contradicción con $t_m \neq t'_m$. Por tanto, $t_1 = t'_1, \dots, t_k = t'_k$.

Entonces, como $\eta = f t_1 \dots t_k = \eta' \zeta$ y $\zeta = \emptyset$.

- (b) Fórmulas: Sea η una fórmula y $\eta = \eta' \zeta$ con η' una fórmula. Demostramos por inducción en la longitud de η que $\eta = \eta'$ y $\zeta = \emptyset$. Si $|\eta| = 1$, como η' es una fórmula, tenemos que $0 < |\eta'| \leq |\eta| = 1$, luego $|\eta'| = |\eta| = 1$ y $|\zeta| = 0$.

Supongamos que $|\eta| = l > 1$ y se cumple para toda fórmula de longitud menor a l .

Consideremos primero el caso de fórmulas de complejidad 0. Digamos que $\eta = Rt_1 \dots t_m$ con R símbolo de relación m -ario ($\doteq$ y $m = 2$). Como $\eta = \eta'\zeta$, concluimos que el primer símbolo de η' es R . Como η' es una fórmula, tenemos que $\eta' = Rt'_1 \dots t'_m$. Entonces, $t_1 \dots t_m = t'_1 \dots t'_m \zeta$. Ahora bien, por el caso de términos ya demostrado, obtenemos que $t_1 = t'_1, \dots, t_m = t'_m$ y $\zeta = \emptyset$. Por tanto, $\eta = \eta'$.

Consideremos $\eta = \neg\varphi$. Como $\eta = \eta'\zeta$, concluimos que el primer símbolo de η' es $\neg$. Por tanto, como η' es una fórmula, $\eta' = \neg\phi$ con ϕ fórmula. Entonces, $\varphi = \phi\zeta$ con $|\varphi| < l$. Entonces, por hipótesis de inducción, $\varphi = \phi$ y $\zeta = \emptyset$, luego $\eta = \eta'$.

Consideremos $\eta = \exists x\varphi$. Como $\eta = \eta'\zeta$, concluimos que los dos primeros símbolos de η' son $\exists x$. Por tanto, como η' es una fórmula, $\eta' = \exists x\phi$ con ϕ fórmula. Entonces, $\varphi = \phi\zeta$ con $|\varphi| < l$. Entonces, por hipótesis de inducción, $\varphi = \phi$ y $\zeta = \emptyset$, luego $\eta = \eta'$.

Finalmente, consideremos $\eta = \varphi_1 \wedge \varphi_2$. Como $\eta = \eta'\zeta$, concluimos que el primer símbolo de η' es $\wedge$. Por tanto, como η' es una fórmula, $\eta' = \wedge\phi_1\phi_2$. O bien $|\varphi_1| \geq |\phi_1|$ o $|\varphi_1| \leq |\phi_1|$. Eliminando los símbolos finales, tenemos que $\varphi_1 = \phi_1\zeta'$ o $\phi_1 = \varphi_1\zeta'$ para alguna palabra ζ' . Como $|\varphi_1| < l$ y $|\phi_1| < l$, concluimos por hipótesis de inducción que $\varphi_1 = \phi_1$. Ahora, como $\varphi_2 = \phi_1\phi_2\zeta$, eliminando los símbolos iniciales, tenemos que $\varphi_2 = \phi_2\zeta$. Como $|\varphi_2| < l$, concluimos que $\varphi_2 = \phi_2$ y $\zeta = \emptyset$ por hipótesis de inducción. En consecuencia, $\eta = \eta'$ y $\zeta = \emptyset$. ■

1.1.4 Variables libres y substitución de variables

1.1.4.1 Substitución en términos

Definición 1.1.12 (Substitución en términos). Sean t y τ términos y x una variable simple. El término $\text{Sub}_x^T[t]$ dada por *sustituir* x por τ en t se define recursivamente respecto a la complejidad sintáctica de t .

(i) En el caso de complejidad 0, consideramos dos posibles situaciones:

Si $t = x$, definimos $\text{Sub}_x^T[t] = \tau$;

Si $t \neq x$, definimos $\text{Sub}_x^T[t] = t$.

(ii) En el caso de complejidad $\chi > 0$, si $t = ft_1 \dots t_k$, definimos

$$\text{Sub}_x^T[t] = f \text{Sub}_x^T[t_1] \dots \text{Sub}_x^T[t_k].$$

Ejemplo 1.1.3. Consideremos el término $p(x, y) = x^2 + 2xy + y^2$. Dado el término $q(x) = x + 1$, podemos considerar el término por sustitución $p(x, q(x)) = x^2 + 2x(x + 1) + (x + 1)^2$.

Lema 1.1.2. Sean t y τ términos y x una variable simple. Supongamos que $t \in \text{Ter}_w^L$ es un término de complejidad ξ y $\tau \in \text{Ter}_w^L$ es un término de complejidad ζ . Entonces, $\text{Sub}_x^T[t] \in \text{Ter}_{(w \setminus \{x\}) \cup w_\tau}^L$ es un término de complejidad χ con $\xi \leq \chi \leq \xi + \zeta$.

Demostración: Lo demostramos por inducción en la complejidad de t .

Si t tiene complejidad 0, entonces, $t = x$ o $t \neq x$. Si $t = x$, entonces, $\text{Sub}_x^T[t] = \tau$ y el lema se cumple trivialmente. Si $t \neq x$, entonces, $\text{Sub}_x^T[t] = t$ y el lema se cumple.

Supongamos que t tiene complejidad $\xi > 0$ y el lema se cumple para términos de complejidad menor a ξ . En ese caso, $t = ft_1 \dots t_k$ con f símbolo de función k -ario y $t_1, \dots, t_k \in \text{Ter}_w^L$ términos de complejidad menor a ξ . Digamos que ξ_i es la complejidad de t_i para cada i . Entonces, $\xi = \max_i \xi_i + 1$ por Teorema 1.1.8. Por definición, $\text{Sub}_x^T[t] = f \text{Sub}_x^T[t_1] \dots \text{Sub}_x^T[t_k]$. Por hipótesis de inducción, $\text{Sub}_x^T[t_i] \in \text{Ter}_{(w \setminus \{x\}) \cup w_\tau}^L$. Por último, digamos que χ_i es la complejidad de $\text{Sub}_x^T[t_i]$ para cada i . En ese caso, por Teorema 1.1.8, la complejidad de $\text{Sub}_x^T[t]$ es $\chi = \max_i \chi_i + 1$. Por hipótesis de inducción, $\xi_i \leq \chi_i \leq \xi_i + \zeta$, luego $\xi \leq \chi \leq \xi + \zeta$. ■

1.1.4.2 Substitución en fórmulas

Ahora bien, en el caso de fórmulas, la definición $\text{Sub}_x^\tau[\varphi]$ no es tan trivial. El problema radica en el caso de la cuantificación. En efecto, para definir $\text{Sub}_x^\tau[\exists y \varphi]$ debemos distinguir dos posibles situaciones excepcionales: cuando $y = x$ y cuando y aparece en τ . Puesto que debemos interpretar las variables ligadas como ficticias (es decir, como si no apareciesen), solo debemos sustituir las apariciones libres de x . Por tanto, el caso $y = x$ debemos ignorarlo. Por otro lado, τ debe sustituir a x como "variable libre". En consecuencia, si y aparece en τ , debemos primero renombrar y como una nueva variable z para mantener τ como "libre" en φ .

Definición 1.1.13 (Sustitución en fórmulas). Sean φ una fórmula, τ un término y x una variable simple. Fijamos un subconjunto finito v de variables que contiene todas las variables que aparecen en φ , τ y x . La fórmula $\text{Sub}_x^\tau[\varphi]_v$ dada por *sustituir* x por τ en φ se define recursivamente respecto a la complejidad sintáctica de φ .

- (i) Para $\varphi = Rt_1 \dots t_m$ (incluyendo $=$ y $\top$), definimos $\text{Sub}_x^\tau[\varphi]_v = R \text{Sub}_x^\tau[t_1] \dots \text{Sub}_x^\tau[t_m]$.
- (ii) Si $\varphi = \neg\psi$, definimos $\text{Sub}_x^\tau[\varphi]_v = \neg \text{Sub}_x^\tau[\psi]_v$.
- (iii) Si $\varphi = \psi_1 \wedge \psi_2$, definimos $\text{Sub}_x^\tau[\varphi]_v = \text{Sub}_x^\tau[\psi_1]_v \wedge \text{Sub}_x^\tau[\psi_2]_v$.
- (iv) Si $\varphi = \exists y\psi$, consideramos tres casos:
 - Si y no aparece en τ e $y \neq x$, definimos $\text{Sub}_x^\tau[\varphi]_v = \exists y \text{Sub}_x^\tau[\psi]_v$.
 - Si y no aparece en τ pero $y = x$, definimos $\text{Sub}_x^\tau[\varphi]_v = \varphi$.
 - Si y aparece en τ , definimos $\text{Sub}_x^\tau[\varphi]_v = \exists z \text{Sub}_x^\tau[\text{Sub}_y^z[\psi]_{v,z}]$, donde $z := z_v$ no está en v .

Observación 1.1.14. La definición de sustitución presupone que disponemos de una función $v \mapsto z_v$ que nos proporciona una nueva variable z_v para cada conjunto finito de variables v . Ignoremos por ahora los detalles técnicos sobre la definición de dicha función.

Notación. Escribimos simplemente $\text{Sub}_x^t[n]$ para $\text{Sub}_x^t[n]_v$ con v el mínimo conjunto de variables que contiene todas las variables que aparecen en τ , φ y x .

Ejemplo 1.1.4. Si sustituimos z por $p(x, y) = x^2 + y^2$ en la fórmula $\varphi(y, z) = \exists x \, x^2 + y = z$, obtenemos

$$\varphi(y, p(x, y)) = \exists w \, w^2 + y = x^2 + y^2$$

donde w es una variable nueva.

Lema 1.1.3. Sean φ una fórmula, τ un término y x una variable simple. Sea v un conjunto finito de variables que contiene todas las variables que aparecen en φ , τ y x . Supongamos que $\varphi \in \text{For}^u \mathbf{L}$ es una fórmula de complejidad χ y $\tau \in \text{Ter}^w \mathbf{L}$ es un término. Entonces, $\text{Sub}_x^\tau[\varphi]_v \in \text{For}^{(u \setminus x) \cup w} \mathbf{L}$ es una fórmula de complejidad χ . Además:

- (1) Si x no es variable de τ y no aparece ligada en φ , entonces, x no aparece en $\text{Sub}_x^\tau[\varphi]_v$.
- (2) Si las variables de τ no aparecen ligadas en φ , entonces, $\text{Sub}_x^\tau[\varphi]_v$ es independiente de v .
- (3) Si las variables de τ no aparecen ligadas en φ , $\text{Sub}_x^\tau[\varphi]_v$ y φ tienen las mismas variables ligadas.

Demostración: Lo demostramos por inducción en la complejidad de φ .

Si φ tiene complejidad 0, entonces, $\varphi = Rt_1 \dots t_m$ (incluyendo $=$ y $\top$). Por definición,

$\text{Sub}_x^\tau[\varphi]_v = R \text{Sub}_x^\tau[t_1] \dots \text{Sub}_x^\tau[t_m]$. Por Lema

1.1.13

, concluimos que $\text{Sub}_x^\tau[\varphi]_v \in \text{For}^{(v \setminus x) \cup v} \mathbf{L}$ es una fórmula de complejidad 0. Además, si x no aparece en τ , concluimos que x no aparece en $\text{Sub}_x^\tau[\varphi]_v$. Por otro lado, en este caso, φ y $\text{Sub}_x^\tau[\varphi]_v$ no tienen variables ligadas y $\text{Sub}_x^\tau[\varphi]_v$ no depende de v .

Supongamos que φ tiene complejidad $\chi > 0$ y el lema se cumple para fórmulas de complejidad menor a χ . En ese caso, o $\varphi = \neg\phi$, $\varphi = \psi_1 \wedge \psi_2$ o $\varphi = \exists y\phi$. En los dos primeros casos, $\text{Sub}_x^\tau[\varphi]_v = \neg\text{Sub}_x^\tau[\phi]_v$ o $\text{Sub}_x^\tau[\varphi]_v = \wedge \text{Sub}_x^\tau[\psi_1]_v \text{Sub}_x^\tau[\psi_2]_v$, y concluimos por hipótesis de inducción. Consideremos el caso $\varphi = \exists y\phi$.

Si $y = x$ e y no aparece en τ , tenemos que $\text{Sub}_x^\tau[\varphi]_v = \varphi$ por definición, y el enunciado se cumple trivialmente —nótese que x no aparece libre en φ en este caso—. Si $y \neq x$ e y no aparece en t , tenemos que $\text{Sub}_x^\tau[\varphi]_v = \exists y \text{Sub}_x^\tau[\phi]_v$ y concluimos por hipótesis de inducción. Si y aparece en τ , tenemos que $\text{Sub}_x^\tau[\varphi]_v = \exists z \text{Sub}_x^\tau[\text{Sub}_y^\tau[\phi]_{v,z}]_{v,z}$, donde $z := z_v$ es una variable simple que no aparece en v . Por hipótesis de inducción, $\text{Sub}_x^\tau[\text{Sub}_y^\tau[\phi]_{v,z}]_{v,z} \in \text{For}^{(v \setminus \{x,y\}) \cup v \cup z} \mathbf{L}$ es una fórmula de igual complejidad que ϕ . Por tanto, $\text{Sub}_x^\tau[\varphi]_v \in \text{For}^{(v \setminus x) \cup v} \mathbf{L}$ es una fórmula de igual complejidad que ϕ . Como z no aparece en φ , $\text{Sub}_y^\tau[\phi]_{v,z}$ y ϕ tienen las mismas variables ligadas. Si x no aparece ligada en φ y no aparece en τ , concluimos por hipótesis de inducción que x no aparece en $\text{Sub}_x^\tau[\varphi]_v$. Como y aparece ligada en φ , no es necesario comprobar que $\text{Sub}_x^\tau[\varphi]_v$ y φ tienen las mismas variables ligadas (de hecho, no las tienen pues $\text{Sub}_x^\tau[\varphi]_v$ tiene variable ligada z , la cual no aparece en φ) o es independiente de v (de hecho, no lo es pues la elección de z depende de v). ■

Observación 1.1.15. Un caso particular de sustitución de variables es cuando el término t es también una variable, es decir, cuando se sustituyen variables por variables. Este caso particular se denomina *cambio de variables*. Dada una variable $x = \{x_i\}_{i \in I}$, una *copia* de x es otra variable $y = \{y_i\}_{i \in I}$ junto con una biyección $\sigma : x_i \mapsto y_i$ entre las dos variables x e y . Para una variable ordenada $\bar{x} = (x_i)_{i \in I}$, una *copia* $\bar{y} = (y_i)_{i \in I}$ es otra variable ordenada de igual longitud. Una copia $\bar{y} = (y_1, \dots, y_n)$ de una variable ordenada $\bar{x} = (x_1, \dots, x_n)$ induce las correspondencias $\text{Sub}_{x_1}^{y_1} \dots \text{Sub}_{x_n}^{y_n} : \text{Ter}^x \mathcal{L} \rightarrow \text{Ter}^y \mathcal{L}$ y $\text{Sub}_{x_1}^{y_1} \dots \text{Sub}_{x_n}^{y_n} : \text{For}^x \mathcal{L} \rightarrow \text{For}^y \mathcal{L}$ dadas por sustituir $\bar{x}$ por $\bar{y}$ en el orden fijado.

Notación: Sea $t(x_1, \dots, x_n)$ un término con variables en $x_1, \dots, x_n$ y $t_1, \dots, t_n$ términos. Escribimos $t(t_1, \dots, t_n)$ para indicar que se sustituyen simultáneamente $x_1, \dots, x_n$ por $t_1, \dots, t_n$,

es decir:

$$t(t_1, \dots, t_n) := \text{Sub}_{w_1}^{t_1} [\dots [\text{Sub}_{w_n}^{t_n} [\text{Sub}_{x_1}^{w_1} [\dots [\text{Sub}_{x_n}^{w_n} [t(x_1, \dots, x_n)]] \dots]] \dots]]$$

donde $w_1, \dots, w_n$ son ciertas variables accesorias nuevas que no aparecen ni en $t_1, \dots, t_n$ ni en t .

Del mismo modo, para una fórmula $\varphi(x_1, \dots, x_n)$ con variables libres en $x_1, \dots, x_n$, escribimos $\varphi(t_1, \dots, t_n)$ para

$$\varphi(t_1, \dots, t_n) := \text{Sub}_{w_1}^{t_1} [\dots [\text{Sub}_{w_n}^{t_n} [\text{Sub}_{x_1}^{w_1} [\dots [\text{Sub}_{x_n}^{w_n} [\varphi(x_1, \dots, x_n)]] \dots]] \dots]]$$

donde $w_1, \dots, w_n$ son ciertas variables accesorias nuevas que no aparecen ni en $t_1, \dots, t_n$ ni en φ .

1.2 Estructuras

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.